

THE RENAISSANCE DEVELOPER ACADEMY

Curriculum Outline v1.0

An Intensive 10-Week AI-Augmented Bootcamp
Transforming Mid-Level Engineers into Polymath Builders

Format: Full-Time Intensive (8–12 weeks)
Delivery: External Training Academy
Assessment: Capstone Projects + Stackable Digital Credentials

DRAFT — March 2026

1. Program Overview

The Renaissance Developer Academy is a 10-week full-time intensive bootcamp designed to transform mid-level software engineers (3–7 years experience) into polymath builders who can operate as forward-deployed engineers, bridging code, business, people, and AI into a single craft.

The program is built on a core philosophy: AI tools have shifted the core engineering competency from writing code to architecting, validating, integrating, and delivering systems within business context. We use AI to teach AI — every tool, every workflow, every assignment is AI-augmented from Day 1.

1.1 Target Audience

- Mid-level software engineers with 3–7 years of professional experience
- Comfortable writing production code in at least one language
- Eager to expand from specialist to generalist/polymath
- Motivated to operate at the intersection of engineering and business

1.2 Program Outcomes

Upon completion, graduates will be able to:

1. Ship full-stack production applications in days using AI-augmented workflows
2. Design multi-component systems with documented architecture decisions
3. Translate ambiguous business problems into validated technical solutions
4. Lead AI-augmented development workflows and evaluate AI-generated code critically
5. Communicate technical decisions to any audience, from engineers to executives
6. Operate a complete GitHub-based CI/CD pipeline from commit to production
7. Understand autonomous agent architectures through hands-on work with the OpenClaw ecosystem

1.3 Pedagogical Philosophy

Exercise-first learning: 75% hands-on practice, 25% theory. Concepts are introduced through doing, not lectures.

Flipped classroom: Pre-work covers foundational concepts (via NotebookLM study materials, audio overviews, and flashcards); live time is for problem-solving, design reviews, and project work.

AI-native workflow: AI tools (Claude Code CLI, Kiro, Antigravity, OpenClaw) are the development environment from Day 1 — not a module tacked on at the end. Students learn to be architects and editors, not just coders.

Business context in every project: Every technical deliverable requires a Lean Canvas, user validation, or business metrics. The question is never just “does it work?” but “who does it serve and why does it matter?”

2. Tooling Ecosystem

The curriculum uses five AI-powered development tools, anchored by GitHub as the universal backbone for version control, CI/CD, project management, and collaboration. No specific cloud platform or project management tool is mandated.

2.1 GitHub as the Backbone

GitHub serves as the single source of truth across the entire program. All code, documentation, CI/CD pipelines, and project tracking flow through GitHub.

2.1.1 Version Control & Collaboration

- Every student maintains a personal GitHub portfolio repository from Week 1
- All projects use feature branches, pull requests, and code review workflows
- Students practice conventional commits, branch naming conventions, and merge strategies (rebase vs. squash vs. merge)
- Collaborative projects require cross-review: every PR must be reviewed by at least one peer before merge

2.1.2 CI/CD with GitHub Actions

GitHub Actions is the mandated CI/CD platform. Students build progressively complex pipelines:

- **Week 2:** Basic workflows — lint, test, build on push/PR events
- **Week 4:** Matrix builds across multiple Node/Python versions, dependency caching, artifact publishing
- **Week 6:** Docker image builds, container registry publishing, multi-environment deployment (staging/production)
- **Week 8:** Reusable workflows, GitHub Environments with approval gates, secrets management, and security scanning
- **Week 10:** Agentic workflows (GitHub’s Continuous AI preview) — AI agents that auto-triage issues, update docs, and monitor test coverage within Actions

2.1.3 Project Management with GitHub Projects

No external project management tool is mandated. Students are encouraged to use GitHub’s built-in project management features:

- **GitHub Issues:** Bug tracking, feature requests, and task management with labels, assignees, and milestones
- **Sub-issues:** Parent-child task hierarchies for breaking down complex work into manageable pieces
- **GitHub Projects:** Kanban boards and spreadsheet-style table views for sprint planning, filtering, sorting, and grouping issues
- **Automation:** Auto-move issues across columns when PRs are merged, auto-label based on file changes

Students who prefer external tools (Linear, Jira, Notion, etc.) are free to use them, but must still maintain their GitHub Issues and PRs as the primary record of work.

2.1.4 GitHub Skills Session

A dedicated 2-day session in Week 1 covers GitHub from foundations to advanced usage:

Day 1 — Foundations (for those who need it):

- Git mental model: commits as snapshots, branches as pointers, HEAD and detached HEAD
- Repository setup, .gitignore, README conventions, LICENSE selection
- Branching strategies: GitHub Flow (recommended), Git Flow (when appropriate), trunk-based development
- Pull request anatomy: description templates, reviewers, linked issues, draft PRs
- Merge strategies: when to squash, rebase, or merge commit
- Conflict resolution: hands-on exercises with realistic merge conflicts

Day 2 — Advanced GitHub & CI/CD:

- GitHub Actions fundamentals: workflows, triggers, jobs, steps, runners, and the YAML schema
- Writing your first CI pipeline: checkout, install, lint, test, build
- Secrets management and environment variables
- GitHub Projects setup: creating boards, custom fields, automation rules
- GitHub MCP Server: using AI agents (Copilot, Claude Code) to manage issues and PRs programmatically
- GitHub CLI (gh): managing repos, issues, PRs, and workflows from the terminal
- GitHub Codespaces: cloud development environments for consistent team setups
- Branch protection rules, required reviews, and status checks

2.2 AI Development Tools

Tool	Primary Role	Key Capability	Curriculum Use
------	--------------	----------------	----------------

Claude Code CLI	Terminal-native coding agent	Multi-agent parallelism, 1M token context, Unix composability	Daily development tool, CI/CD integration, code review workflows
Amazon Kiro	Spec-driven IDE	Requirements → design → tasks → code pipeline, Agent Hooks	Architecture modules, spec-driven development, requirements engineering
Google Antigravity	Multi-agent IDE	Manager View for 5+ parallel agents, artifact-based audit trail	Complex prototyping, multi-agent orchestration, design sprints
OpenClaw + ecosystem	Autonomous AI agent	Local-first, 50+ integrations, extensible skills, messaging-based UI	Full-stack case study, agent architecture, security analysis, DevOps automation
Google NotebookLM	AI study companion	Audio overviews, flashcards, quizzes, Learning Guide mode, mind maps	Pre-work study, self-assessment, concept review, research companion

2.3 OpenClaw Ecosystem as Full-Stack Case Study

Rather than mandating a specific cloud platform, the curriculum uses the OpenClaw ecosystem as a comprehensive full-stack development case study. Students study, deploy, extend, and build with increasingly sophisticated variants:

Project	Complexity	What Students Learn	Stack	Curriculum Phase
Nanobot	~4K lines Python	Agent fundamentals: tool use, memory, messaging	Python, local LLMs	Week 2–3
NanoClaw	~5 files, containers	Container isolation, security-first agent design, Claude Code integration	Docker, Node.js	Week 4–5
ClawSwarm	Multi-agent, Rust	Multi-agent orchestration, gRPC, unified messaging architecture	Rust, gRPC, Docker	Week 6–7
OpenClaw	430K+ lines, monorepo	Enterprise-scale architecture, plugin systems, security analysis, ClawHub ecosystem	Node.js, WebSocket	Week 8–10

Why this progression works: Students start with a codebase small enough to read in an afternoon (Nanobot), learning how autonomous agents actually work. They graduate to container-isolated agents (NanoClaw) where security and architecture become primary

concerns. Multi-agent orchestration (ClawSwarm) introduces distributed systems thinking. Finally, analyzing the full OpenClaw monorepo teaches enterprise-scale code navigation, plugin architecture, and the real-world security challenges of open-source AI systems. This mirrors the progression from prototyping to production that defines the Renaissance Developer role.

Cloud-agnostic deployment: Students deploy their agents to whichever cloud platform they choose (AWS, GCP, Azure, DigitalOcean, or even a Raspberry Pi). The emphasis is on containerized deployment (Docker), infrastructure-as-code principles, and CI/CD via GitHub Actions — skills that transfer to any cloud provider.

3. Curriculum Structure: 10-Week Overview

3.1 Weekly Rhythm

Each week follows a consistent structure designed to balance learning, building, and reflection:

- **Monday–Thursday:** Structured learning blocks (1–2 hrs concept introduction → 2–3 hrs individual/pair exercises → 3–4 hrs project work and design reviews)
- **Friday:** Design reviews, student presentations, retrospectives, and “Impossible Stuff Day” (self-directed exploration inspired by the Recurse Center)
- **Daily:** 30-minute stand-up and reflection check-in
- **Pre-work:** NotebookLM study materials assigned each weekend for the following week

3.2 Phase Map

Phase	Theme	Primary Tools	Key Deliverables
Week 1	Foundations: GitHub, AI Tools & Mindset Reset	GitHub, Claude Code CLI, NotebookLM	GitHub portfolio repo, CI pipeline, tool proficiency baseline
Week 2	AI-Augmented Development & Agent Fundamentals	Claude Code CLI, Nanobot, NotebookLM	Working agent (Nanobot), AI code review log, prompt library
Week 3	Full-Stack Development at Speed	Antigravity, Claude Code, GitHub Actions	Full-stack app (React + API + DB), deployed with CI/CD
Week 4	System Design & Architecture	Kiro, NanoClaw, GitHub Actions	Architecture Decision Records (ADRs), spec-driven app, containerized agent
Week 5	Data, Integration & Reliability	Kiro, Claude Code, GitHub Actions	Data pipeline, SLO dashboard, incident response runbook
Week 6	Rapid Prototyping & Design Sprints	Antigravity, ClawSwarm, all tools	48-hour hackathon deliverable, user-tested prototype, multi-agent system
Week 7	Business Immersion & Product Thinking	NotebookLM, Kiro, Claude Code	Lean Canvas, customer discovery report, business-justified technical proposal
Week 8	Communication, Leadership & Enterprise Scale	OpenClaw (full), all tools	Technical spec, executive presentation, security audit of OpenClaw
Week 9	Capstone Build Sprint	All tools, student choice	Capstone project: working product, deployed, documented, user-validated

Week 10	Capstone Delivery & Graduation	All tools	Final presentation, portfolio, peer reviews, credential ceremony
----------------	--------------------------------	-----------	--

4. Module Details

Each week is a self-contained module with clear learning objectives, exercises, and deliverables. Modules are designed to be modular but structural — they build on each other, but each can also serve as a standalone intensive for targeted upskilling.

Module 1: Foundations — GitHub, AI Tools & Mindset Reset

Week 1 | *The ground rules: how Renaissance Developers work*

Learning Objectives:

1. Master GitHub as the backbone: repos, branches, PRs, Actions, Projects, CLI
2. Set up and navigate all five AI development tools
3. Understand the “Creative Director” mindset: architect + editor, not just coder
4. Build a personal NotebookLM notebook with all course materials for ongoing study

Key Sessions:

- Day 1–2: GitHub Skills Session (see Section 2.1.4 above)
- Day 3: AI Tool Setup Lab — install and configure Claude Code CLI, Kiro, Antigravity; connect to GitHub
- Day 4: First AI-Augmented Build — build a complete web app in one session using Claude Code CLI, push to GitHub, trigger a CI pipeline
- Day 5: NotebookLM Mastery + Reflection — create study notebooks, generate audio overviews and flashcards for Week 2 pre-work

Deliverables:

- Personal GitHub portfolio repository with README, CI/CD pipeline (lint + test + build), and GitHub Projects board
- AI tool proficiency baseline self-assessment
- First blog post draft: “What I Expect to Learn” (published to repo wiki)

Badge: *Renaissance Foundations*

Module 2: AI-Augmented Development & Agent Fundamentals

Week 2 | *AI is your junior developer — fast but needs supervision*

Learning Objectives:

1. Develop systematic prompt engineering skills for code generation
2. Build critical evaluation skills for AI-generated code (correctness, security, performance, maintainability)

3. Understand autonomous agent architecture by studying and extending Nanobot (~4K lines)
4. Practice AI-augmented code review workflows in GitHub PRs

Key Sessions:

- Prompt Engineering Workshop: systematic approaches to getting better code from AI (context setting, constraint specification, iterative refinement)
- AI Bug Safari: review 20 AI-generated code samples, identify and document subtle bugs, security vulnerabilities, and performance issues
- Nanobot Deep Dive: fork, read, understand, and extend the Nanobot agent — add a custom skill, connect a new messaging platform
- AI Code Review Lab: practice reviewing AI-generated PRs using a structured checklist

Deliverables:

- Personal prompt library (organized by task type) in GitHub repo
- AI Bug Log: 10 documented cases of AI code failures with root cause analysis
- Working Nanobot agent with a custom skill deployed locally
- GitHub Actions pipeline enhanced with automated testing of AI-generated code

Badge: *AI-Augmented Developer*

Module 3: Full-Stack Development at Speed

Week 3 | *Build end-to-end, deploy by Friday*

Learning Objectives:

1. Ship a complete full-stack application (frontend + API + database) in one week
2. Use Antigravity's multi-agent Manager View to parallelize development tasks
3. Implement a complete CI/CD pipeline with matrix builds, caching, and multi-environment deployment via GitHub Actions
4. Practice cloud-agnostic containerized deployment with Docker

Key Sessions:

- Full-Stack Speed Run: build a React + Node.js/Python API + PostgreSQL app from zero to deployed in 2 days using Antigravity for parallel frontend/backend development
- GitHub Actions Advanced: matrix builds, dependency caching, Docker image publishing, deployment to staging and production environments
- Database fundamentals at speed: schema design, migrations, query optimization — with AI assistance but human judgment
- Cloud Deployment Lab: containerize with Docker, deploy to student's chosen cloud platform

Deliverables:

- Full-stack application deployed to production with automated CI/CD pipeline
- GitHub Actions workflow: lint → test → build → Docker push → deploy (staging + production)
- Architecture walkthrough document in repo wiki

Badge: *Full-Stack Builder*

Module 4: System Design & Architecture

Week 4 | *Think before you code — specs drive everything*

Learning Objectives:

1. Design multi-component systems with documented trade-off analysis
2. Master Kiro's spec-driven development workflow: requirements → design → tasks → code
3. Write Architecture Decision Records (ADRs) and technical specifications
4. Deploy and analyze a containerized agent (NanoClaw) to understand security-first architecture

Key Sessions:

- System Design Workshop: progressive complexity from single-service to distributed systems; live whiteboard sessions with peer critique
- Kiro Spec-Driven Lab: use Kiro to generate formal requirements (EARS notation), design documents, and task breakdowns before writing any code
- ADR Workshop: write Architecture Decision Records for real design choices, using the “context → decision → consequences” template
- NanoClaw Security Lab: deploy NanoClaw in containers, analyze its security model, identify attack surfaces, compare with OpenClaw's default configuration

Deliverables:

- 3 system design documents (progressively complex) with trade-off analysis
- Spec-driven application built entirely through Kiro's workflow
- NanoClaw deployment with written security analysis
- 3 ADRs covering real architectural decisions

Badge: *Systems Architect*

Module 5: Data, Integration & Reliability

Week 5 | *Build systems that survive contact with reality*

Learning Objectives:

1. Design and implement data integration patterns (ETL, event-driven, API gateway)
2. Define and implement SLOs/SLIs for a real service
3. Practice incident response and write post-incident reviews
4. Build monitoring and alerting into CI/CD pipelines

Key Sessions:

- Data Integration Patterns: hands-on with ETL pipelines, event-driven architectures, API gateways — when to use which
- SRE Fundamentals: defining SLOs, implementing SLIs, error budgets, and the relationship between reliability and feature velocity
- Chaos Engineering Lab: introduce failures into a running system, observe behavior, write incident response documentation
- Monitoring & Observability: integrate logging, metrics, and tracing into existing projects; build a dashboard

Deliverables:

- Data pipeline connecting 2+ services with documented integration pattern choices
- SLO dashboard for a production service
- Incident response runbook + post-incident review from the chaos engineering exercise
- GitHub Actions monitoring: alerts on deployment failures, test coverage regressions

Badge: *Reliable Systems Engineer*

Module 6: Rapid Prototyping & Design Sprints

Week 6 | 48 hours from problem to working prototype

Learning Objectives:

1. Apply design sprint methodology (define → ideate → prototype → test) under time pressure
2. Use Antigravity's multi-agent orchestration for ambitious-scope rapid builds
3. Build and orchestrate a multi-agent system using ClawSwarm
4. Run user validation sessions with real stakeholders

Key Sessions:

- Design Sprint Kickoff: problem statements sourced from real business challenges; teams form and plan
- 48-Hour Hackathon (Tue–Wed): build a working prototype using all available AI tools; Antigravity Manager View for parallel agent development
- ClawSwarm Multi-Agent Lab: configure and deploy a multi-agent system with specialized roles (research, coding, testing) communicating via unified messaging

- User Testing Workshop (Thu): structured user validation sessions with prepared interview guides
- Demo Day (Fri): present prototypes, share learnings, vote on “most likely to ship”

Deliverables:

- Working prototype built in 48 hours, user-tested, with documented feedback
- ClawSwarm multi-agent configuration with at least 2 specialized agents
- User testing report with insights and iteration priorities
- Retrospective: what worked, what didn’t, what you’d do differently

Badge: *Rapid Builder*

Module 7: Business Immersion & Product Thinking

Week 7 | *The question is never just “does it work?” but “who does it serve?”*

Learning Objectives:

1. Create Lean Canvases and value propositions for technical products
2. Conduct customer discovery interviews and synthesize findings
3. Read and interpret P&L statements, unit economics, and KPI dashboards
4. Write business cases that justify technical investments to non-technical leaders

Key Sessions:

- Lean Canvas Workshop: every project from this point forward requires a Lean Canvas before any code is written
- Customer Discovery Lab: interview real potential users (sourced from partner organizations); synthesize findings using Jobs-to-be-Done framework
- Finance for Engineers: reading P&L statements, calculating ROI, understanding unit economics — the numbers that executives care about
- Business Case Writing: take a technical project and write a 1-page business justification that would convince a CFO to fund it
- Guest Speaker: forward-deployed engineer or technical product leader from industry

Deliverables:

- Lean Canvas for capstone project (required before capstone build begins)
- Customer discovery report: 5+ interview syntheses with Jobs-to-be-Done analysis
- Business case document for a technical investment proposal
- NotebookLM Deep Research: industry analysis relevant to capstone project

Badge: *Product-Minded Engineer*

Module 8: Communication, Leadership & Enterprise Scale

Week 8 | Bridge any audience gap — from C-suite to command line

Learning Objectives:

1. Communicate technical decisions to any audience (engineers, executives, customers)
2. Navigate enterprise-scale codebases by analyzing the full OpenClaw monorepo
3. Conduct a security audit of a complex open-source system
4. Write technical specifications and executive summaries for the same project

Key Sessions:

- Multi-Audience Communication Workshop: present the same technical decision to three audiences (engineering team, VP of Product, board of directors); receive peer feedback on each
- OpenClaw Monorepo Deep Dive: navigate, analyze, and document a 430K+ line open-source codebase; understand its monorepo structure, plugin architecture, and Gateway/Agent/CLI separation
- Security Audit Lab: conduct a structured security assessment of OpenClaw — identify vulnerabilities, evaluate ClawHub skill vetting, assess prompt injection risks, and write a security report
- Technical Writing Workshop: write a formal technical specification and an executive summary of the same project; practice Architecture Decision Records at enterprise scale
- Leadership Simulation: role-play leading a cross-functional technical decision with competing stakeholder interests

Deliverables:

- Technical specification for a system improvement (real OpenClaw or capstone-related)
- Executive summary of the same system — tailored for non-technical leadership
- OpenClaw security audit report with findings and recommendations
- Recorded presentation showing multi-audience adaptation skills

Badge: *Technical Leader*

Module 9–10: Capstone Build, Delivery & Graduation

Weeks 9–10 | *Everything comes together*

Learning Objectives:

1. Deliver a complete product end-to-end: from business problem to deployed, documented, user-validated solution
2. Demonstrate mastery of AI-augmented workflows, system design, rapid delivery, and business thinking
3. Present work at portfolio quality — ready for job interviews, client demos, or conference talks

Week 9 — Build Sprint:

- Full-time capstone development using all tools and techniques from the program
- Daily stand-ups with instructors for guidance and unblocking
- Mid-week design review: architecture, progress, pivot decisions
- User validation sessions with real users
- Complete CI/CD pipeline through GitHub Actions (including automated testing, security scanning, and deployment)

Week 10 — Delivery & Graduation:

- Monday–Tuesday: Final polish, documentation, portfolio preparation
- Wednesday: Peer reviews (structured code review + product feedback using a rubric)
- Thursday: Capstone presentations — mock “investor pitch” format (3 minutes demo, 2 minutes business case, 5 minutes Q&A)
- Friday: Graduation ceremony, credential ceremony (digital badges issued via Credly), alumni network launch

Capstone Requirements:

- Working product deployed to production with CI/CD via GitHub Actions
- GitHub repository with clean commit history, PRs, documented ADRs, and a comprehensive README
- Lean Canvas and business justification document
- Technical specification and executive summary
- User validation evidence (interview notes, feedback, iteration log)
- 5-minute recorded demo video for portfolio

Badge: *Renaissance Developer — Program Graduate (Meta Badge)*

5. Credential Architecture

The program uses a three-tier stackable credential system, issued as verifiable Open Badge 3.0 digital credentials through Credly. Each badge requires demonstrated competency through project-based assessment, not just attendance.

5.1 Skill Badges (Tier 1)

Awarded upon completion of individual module deliverables with passing peer and instructor review:

Badge	Awarded After	Key Evidence
Renaissance Foundations	Module 1	GitHub repo + CI/CD pipeline
AI-Augmented Developer	Module 2	AI Bug Log + working agent
Full-Stack Builder	Module 3	Deployed full-stack app
Systems Architect	Module 4	ADRs + spec-driven app
Reliable Systems Engineer	Module 5	SLO dashboard + runbook
Rapid Builder	Module 6	Hackathon prototype + user tests
Product-Minded Engineer	Module 7	Lean Canvas + business case
Technical Leader	Module 8	Security audit + exec presentation

5.2 Proficiency Badges (Tier 2)

Aggregated badges recognizing demonstrated competency across related skill areas:

- **Polymath Builder:** Earned by completing Full-Stack Builder + Systems Architect + Reliable Systems Engineer
- **AI Engineering Specialist:** Earned by completing AI-Augmented Developer + Rapid Builder + demonstrating AI tool integration across all projects
- **Business-Minded Technologist:** Earned by completing Product-Minded Engineer + Technical Leader + demonstrating business context in capstone

5.3 Meta Badge (Tier 3)

Renaissance Developer — Program Graduate: Issued upon completion of all 8 skill badges plus the capstone project with passing evaluation. This badge represents the full Renaissance Developer competency: polymath engineering + AI fluency + business immersion + technical leadership.

All badges are shareable on LinkedIn, embeddable in portfolios, and include verifiable metadata

linking to the specific evidence and criteria.

6. Assessment Framework

Assessment combines continuous formative evaluation with summative capstone evaluation. There are no written exams — all assessment is based on demonstrated work.

6.1 Continuous Assessment (70%)

- Weekly deliverables reviewed by peers and instructors using structured rubrics
- GitHub contribution analytics: commit frequency, PR quality, code review engagement
- Design review participation and quality of feedback given to peers
- AI tool usage sophistication (tracked through prompt libraries and code review logs)

6.2 Capstone Assessment (30%)

- Technical execution: architecture quality, code quality, test coverage, CI/CD pipeline
- Product thinking: problem definition, user validation, business justification
- Communication: technical specification, executive summary, live presentation
- AI integration: evidence of sophisticated AI-augmented workflows, verification rigor

7. Recommended Reading & Resources

Pre-loaded into student NotebookLM notebooks for audio overview generation, flashcard creation, and study:

7.1 Core Reading

- *Designing Data-Intensive Applications* by Martin Kleppmann — system design foundation
- *The Lean Startup* by Eric Ries — build-measure-learn methodology
- *Staff Engineer* by Will Larson — operating at and beyond senior level
- *The Pragmatic Programmer* by Hunt & Thomas — polymath engineering philosophy
- *Team Topologies* by Skelton & Pais — organizational design for engineering

7.2 Supplementary

- *An Elegant Puzzle* by Will Larson — systems of engineering management
- *Inspired* by Marty Cagan — product management for technologists
- *The Goal* by Eliyahu Goldratt — systems thinking and constraint theory
- Gergely Orosz, “The Product-Minded Engineer” (article) — the mindset shift
- Gergely Orosz, “Forward Deployed Engineers” (Pragmatic Engineer newsletter) — the role model

8. Next Steps

This outline establishes the curriculum architecture. The following modules need detailed development:

1. Detailed lesson plans for each module (session-by-session breakdown with timing, exercises, and instructor notes)
2. Exercise bank: specific coding exercises, design challenges, and business case scenarios for each module
3. Assessment rubrics: detailed scoring criteria for each deliverable and the capstone project
4. Instructor guide: teaching notes, facilitation guides, and common student challenges per module
5. NotebookLM course notebooks: pre-built notebooks for each module with curated source materials
6. GitHub template repositories: starter repos for each module with CI/CD templates, README templates, and ADR templates

7. Partner organization pipeline: identify companies willing to provide real business problems for customer discovery (Module 7) and capstone projects
8. Credly badge design and criteria specification: visual design, metadata, and issuance criteria for all 11 badges

End of Curriculum Outline v1.0